# Shared infrastructure for interpreting arithmetic R

October 20, 2026

**Abstract**

The library `Interpretation_of_R` provides the proof infrastructure used by the independent relational and functional interpretations of arithmetic in the original rank-based pure type system. Its main contributions are a sound native checker for annotated terms, structural lemmas for dependent telescopes, fixed-answer negative logic, finite selector observations with internal domain certificates, and constructive selection of witnesses from proved inhabitation. This document explains the contracts between these components and the proof steps that keep finite computation, internal inhabitation, and source arithmetic distinct. The arithmetic interpretation theorems themselves belong to the two child libraries.

## 1 Purpose and mathematical boundary

Both arithmetic developments construct terms of the existing calculus. The syntax is `PTSrk.Syntax.Term nat n`, with finite de Bruijn variables and annotations on every lambda. The judgment is `PTSrk.Typing.typ` instantiated with `PTSrk.Ranks.RankSorts`. Writing its sorts as $U_i$, the defining rules are

$$U_i : U_0, \qquad \frac{\Gamma \vdash A : U_a \quad \Gamma, x : A \vdash B : U_b}{\Gamma \vdash \Pi x : A.B : U_{\max(a+1,b)}}.$$

The library does not introduce arithmetic primitives, a new typing relation, cumulativity, or eta conversion. `Core.Typed` is an abbreviation for the original judgment, and `Core.Realizes A` means that a closed original term has type $A$. The wrappers in `DirectRKernel.v` expose the original typing constructors and beta-congruence lemmas without changing their premises.

The two clients share these tools but have distinct carriers and equality relations. The relational library establishes `Theorem4.theorem4`; the functional library establishes `Theorem7.theorem7` and its effective witness interfaces. Neither imports the other. Neither depends on either set-model development. In particular, the common source countermodel discussed below is not a semantics of the PTS.

## 2 Constructing and checking annotated terms

### 2.1 Named builders and structural transport

`DirectRCommonCodes.v` provides a small named construction language. Its constructors retain sort labels, product domains, and lambda annotations. Assembly resolves names against an explicit environment and returns an optional scoped original term. `close` additionally requires a successful closedness test. The named language is therefore a construction aid; every exported proof concerns the assembled original syntax.

The same module defines the answer type, truth type, arrows, negative connectives, selectors, and a family of inhabited rank-raising types. `DirectRStructural.v` proves that weakening, renaming, substitution, and closed-term lifting interact with these constructions as expected.

These identities matter because a proof assembled under one sequence of binders must still carry the correct annotations when moved under another.

Dependent observations are represented as telescopes. Each telescope field records an annotation in the context of all preceding fields. The formation and abstraction theorems traverse those fields and compute the product rank. `TelescopeArguments.arguments_application` then types an application to an actual list of arguments: each argument is checked against its annotation after substitution of every preceding argument. This prevents treating a dependent telescope as a list of unrelated types. Closed arguments can also be lifted uniformly into an arbitrary context.

### 2.2 The native checker and its exact contract

`DirectREval.v` implements beta evaluation on the original annotated syntax. Head evaluation exposes a sort or product without reducing an unused argument first. Full evaluation also visits product domains and lambda annotations. Every successful elementary step is proved to be an original beta step, and successful conversion comparison yields original definitional equality.

`DirectRCheck.v` infers types by structural recursion on the annotated input term. A numeric fuel parameter bounds only reduction needed to inspect or compare types. Its results distinguish acceptance, a diagnostic rejection, and an undecided result caused by exhausted reduction fuel.

**Theorem 2.1** (Native checking soundness)**.** *For every scoped context $\Gamma$, term $t$, expected type $A$, and fuel bound $f$, acceptance of `RkCheck.run_check f` on these inputs implies $\Gamma \vdash t : A$ in the original calculus. This is the public theorem `RkCheck.run_check_sound`.*

*Proof.* First establish soundness of head inspection and conversion. Induction on the input term proves `infer_sound`, assuming an original well-formed context. The application case is the important step: when an inferred type reduces to $\Pi x : A.B$, original regularity, subject reduction, and product inversion supply a formation proof for $A$. The checker may then convert the argument's inferred type to $A$ and apply the original application rule. A conversion test alone never supplies type formation.

The context checker separately constructs the original, Type-valued well-formedness evidence by recursion on the context. Finally `check_sound` checks formation of the expected type before applying the original conversion rule. Combining these results proves the acceptance theorem. □

The tactic `RkCheck.certify_with` uses this theorem and computes a successful run. Its numeric fuel is a proof-search resource, not a new calculus rule. No completeness or non-typability theorem is claimed for the checker's rejected or undecided outputs. `DirectRSymbolic.v` retains a separate tactic that assembles original typing and beta-equivalence proofs for symbolic indices; it likewise adds no typing rule or trusted conversion oracle.

## 3 Negative logic and positive certificates

The fixed answer and truth types are

$$\bot := \Pi Z : U_0.Z : U_1, \qquad \top := U_0 \to U_0 : U_1.$$

Truth has its identity inhabitant. The logical constructions use

$$\neg_c A := A \to \bot, \quad A \wedge_c B := (A \to B \to \bot) \to \bot,$$
$$\bigvee_c (A_0, \ldots, A_k) := (A_0 \to \bot) \to \cdots \to (A_k \to \bot) \to \bot.$$

`DirectRLogic.v` calls $A$ stable when $\neg_c \neg_c A \to A$ is inhabited. It supplies stability and elimination lemmas for the encoded negative connectives. In particular, `NegativeLogic.flat_eliminate` eliminates a finite disjunction into a stable target. It does not permit elimination into every positive type. The arithmetic translations use this interface to realize classical deduction without importing a native classical axiom.

Positive evidence must be retained separately. The shared invariant constructor has the shape

$$I(G, C) := \Pi Z : U_8.(G \to C \to Z) \to Z,$$

where $G : U_6$ is positive observation-domain evidence and $C : U_8$ is a negative case statement. `positive_invariant_formation` gives this type rank ten, and `positive_invariant_intro` constructs it from the two pieces. In a constructor proof one opens the parent's continuation at a fixed $Z$, eliminates child invariants at that same $Z$, and separately builds the parent's positive evidence and negative cases. This is the mechanism that avoids obtaining positive membership by an illicit classical case split.

Rank raising is also explicit. Set $J_1 = \top$ and $J_{k+1} = J_k \to J_k$; each has a canonical inhabitant. For $A : U_a$, $a \leq k$, and $k \geq 2$, the type $J_{k-1} \to A$ has exactly rank $k$. Abstraction and evaluation at the canonical inhabitant map between the original and raised types. In particular, finite-law elimination uses $J_2 \to A$ to obtain an exact rank-three motive. This construction is needed because there is no cumulativity rule.

# 4 Finite observations represented inside the calculus

## 4.1 Selectors, tables, and relation tests

For a positive external size $q$, the selector carrier is

$$F_q := \underbrace{U_1 \to \cdots \to U_1}_{q \text{ arguments}} \to U_1 : U_1, \qquad e_i := \lambda Z_0 \ldots Z_{q-1}.Z_i \quad (i < q).$$

The number of arguments may grow while the carrier rank remains one: each domain $U_1$ itself has type $U_0$. `DirectRFiniteTables.v` realizes unary tables, proves selector application, and derives capped successor and iteration equations. `DirectRBinaryTables.v` adds nested binary selection. Its `binary_table_action` establishes the equation at every valid pair of indices, for any table whose entries stay in range. `DirectRIterations.v` connects explicit repeated application with the capped finite values.

`DirectRRelationTables.v` uses a matrix of types to encode a finite truth table. A true entry is $\bot$ and a false entry is $\top$; the interpreted relation is the negation of the selected entry. Thus a true relation has the identity $\bot \to \bot$, whereas a proof of a false relation yields $\bot$ after application to the fixed inhabitant of $\top$. `negative_true` and `negative_false` formalize these two directions. The Boolean Horn-law lemmas turn valid finite implications into original PTS inhabitants, including branches with false premises.

### 4.2 Internal membership, not an external exhaustiveness assumption

There is no claim that every raw inhabitant of $F_q$ is a selector. Instead `DirectRPositive.v` defines positive finite membership, specialized in `DirectRFiniteObservations.v` to

$$\delta_q(u) := \Pi Q : F_q \to U_3.Q(e_0) \to \cdots \to Q(e_{q-1}) \to Q(u) : U_4.$$

Each selector has the corresponding projection certificate. In the `FiniteObservations` module, `delta_eliminate` specializes it to an exact rank-three motive and supplies all its selector branches. `delta_successor` proves membership closure under capped successor; `BinaryObservations.delta_binary` proves the analogous result for an arbitrary bounded binary table. To construct output membership, these proofs open the output predicate and specialize input membership to that predicate composed with the table operation.

`DirectRFiniteCertificates.v` packages the initial and successor certificates with their literal annotations for use in observation telescopes. `DirectRFiniteUp.v` constructs the upward-closed test used in relational separation. These are actual PTS terms with original typing proofs, not just native propositions saying a finite algebra has a law.

**Theorem 4.1** (Guarded finite-law realization)**.** *Suppose a $k$-argument predicate on $F_q$ has result rank at most three, and every tuple of valid selector indices has an original proof of that predicate. Then the predicate holds for all $k$ arguments carrying $\delta_q$ certificates. The closed, interleaved-guard version is `GuardedFiniteLaws.guarded_finite_realized`.*

*Proof.* `FiniteLawElimination.delta_eliminate_lower` first raises a lower-rank motive to rank three and lowers the resulting proof by application to the inhabitant of $J_2$. The theorem `delta_eliminate_arguments` iterates that construction over all arguments, reducing the result to the supplied selector-tuple branches. `guarded_intro` introduces each object and its membership guard before running the eliminator on the remaining body. This ordering matters: the entire guarded telescope may have rank five, which is too large for the rank-three membership eliminator. □

The finite branch premise quantifies over all tuples of the given length. Checking a few concrete examples cannot replace it. Conversely, a native arithmetic proof about table entries is only a side condition until the table action, negative relation encoding, and membership elimination lemmas have produced the required internal inhabitant.

## 5 Effective witnesses and the source countermodel

`DirectRDerivationCertificates.v` defines finite trees encoding exactly the original typing rules. Reduction certificates enumerate original one-step successors, including reductions in annotations. Conversion certificates exhibit two paths to a common reduct; completeness uses the original common-reduct characterization of beta conversion.

**Theorem 5.1** (Exact derivation certificates)**.** *`DerivationCertificates.derive_sound` turns successful certificate evaluation into original typing. Conversely, `derive_complete` gives a finite certificate for every original typing derivation.*

These certificates support a different task from the native checker. `DirectRWitnessSearch.v` enumerates all finite certificate trees, scans them for a requested closed type, and proves that an inhabitation proof implies eventual success. `WitnessSearch.witness` has the contract

$$(\exists p.\ \vdash p : A) \ \longrightarrow\ \ \{p : \mathrm{Term}(0) \mid\!\vdash p : A\}.$$

The result uses constructive search over a decidable numeric predicate, with termination justified by the already proved existence theorem. It does not extract an arbitrary witness directly from Rocq's `Prop`, assume a choice axiom, decide unrestricted inhabitation, or provide a practical running-time bound. This is the common machinery behind the functional library's effective proof-term interfaces.

`DirectRCountermodel.v` provides the other shared external construction: $\mathbb{N}$ with one extra point $\omega$, implemented as `option nat`. Successor fixes $\omega$; addition and multiplication return zero when an input is $\omega$. Order is the usual order on naturals, every natural is below $\omega$, and $\omega$ is below nothing. Ground arithmetic, bounded downsets, and comparison with a numeral hold, but order reflexivity fails at $\omega$. Each child library supplies its own source semantics and deduction soundness before using this structure to establish nonfaithfulness. The shared structure alone does not prove a source non-derivability claim.

## 6 Source organization and discussion

The sources form four useful reading groups.

- `DirectRCommonCodes.v`, `DirectRKernel.v`, `DirectREval.v`, `DirectRCheck.v`, and `DirectRSymbolic.v` build and certify original terms.
- `DirectRStructural.v` supplies structural transport, and `DirectRTelescopeArguments.v` handles dependent application. `DirectRLogic.v` and `DirectRPositive.v` distinguish stable negative formulas from positive finite membership.
- The `DirectRFinite*` modules, `DirectRBinaryTables.v`, `DirectRBinaryObservations.v`, `DirectRRelationTables.v`, `DirectRGuardedFiniteLaws.v`, and `DirectRIterations.v` implement the uniform finite-observation constructions.
- The certificate and search modules encode original derivations and select effective witnesses. `DirectRCountermodel.v` supplies the common external source countermodel.

The main design choice is to keep the output contract at original typing throughout. The checker, symbolic tactics, numerical arithmetic, and finite tables all assist proof construction, while their soundness lemmas expose exactly where a PTS inhabitant is produced. Fixed ranks control eliminations; enlarging a finite table changes the size of terms, not the rules of the calculus.

The infrastructure itself asserts neither the arithmetic interpretation theorems nor external uninhabitability of $\bot$. An internal refutation is a term of type $A \to \bot$; concluding that $A$ has no closed inhabitant also needs external nontriviality of the answer type. The two statements must not be conflated. Similarly, successful witness search is effective in the constructive sense above, without a useful complexity claim.

## 7 Reproducing the verification

The Dune library name is `Interpretation_of_R`, independently of its physical directory. The pinned environment is OCaml 4.14.2, Dune 3.23.1, Rocq 9.2.0, Stdlib 9.1.0, and Equations 1.3.2+9.2. Setup and full local CI are documented in the repository's `docs/DirectR-CI.md`.

From the repository root, in that configured environment:

```
dune build theories/Interpretation_of_R/DirectRCheck.vo
```

```
dune build theories/Interpretation_of_R/DirectRWitnessSearch.vo
dune build @theories/Interpretation_of_R/relational/audit
dune build @theories/Interpretation_of_R/functional/audit
```

The two audits check public interfaces and their transitive assumptions. Allocate a fresh absolute Dune build directory when fresh printed assumptions are required; an incremental success may reuse cached output. `bash tools/check-local-ci.sh` additionally runs the complete staged driver, actionlint, and a separate pristine upstream build.

The retained 14 October layout verification includes a fresh complete build, exact public types, closed assumption reports for the arithmetic endpoints, and recursive kernel checks. Its command record is `docs/direct-r-handoff/verification/theories-layout-2026-10-14/COMMANDS.md`. The recursive loaded-context inventory includes the unchanged upstream functional-extensionality dependency; this differs from the closed transitive reports of the arithmetic endpoints. The live handoff records newer checks and their exact scope. Old comments saying an early draft was uncompiled describe its authoring history, not the status of the current checked declarations.